

wasm4pm

A Process-Mining Platform with a Lawful Cognition Layer

Technical Monograph, Release v26.6.10

Sean Chatman

June 10, 2026

Abstract

wasm4pm is a process-mining platform built as two cooperating layers: a Rust core compiled to WebAssembly that hosts 60 process-mining algorithms, and a TypeScript monorepo of twelve packages whose published command-line interface, **wpm**, drives discovery, conformance checking, and predictive analysis from event logs. On top of the mining core sits a *cognition layer* — a “periodic table” of 52 classical symbolic-AI reasoning *breeds*, each reconstructed from its source paper, each gated by an anti-cheat oracle calculus, and each forced through a lawful runtime lifecycle in which every successful run must yield an inference trace, pass postconditions, and replay against an object-centric event-log (OCEL) lifecycle model with conformance fitness exactly 1.0. The platform’s operating doctrine, after van der Aalst, is *process truth*: if the code says it worked but the event log cannot prove a lawful process happened, then it did not work. This monograph documents the entire system at release v26.6.10: architecture, the cognition periodic table and its ten-rung standing ladder, the universal anti-cheat oracle calculus (U1–U6) with 52 oracles and 52 paired adversaries, paper-grounded falsification against 52 fixture files with bibliographic provenance, the OCEL/OCPN process-truth machinery, the BLAKE3 receipt chain, and the release certificate evidencing 60/60 algorithm reachability and 60 positive, 120 negative, and 60 invariant behavior cases. Every numeric claim in this document was verified against the repository at the v26.6.10 working tree before being written down.

Contents

1	Introduction	3
1.1	What wasm4pm is	3
1.2	The doctrine: process truth	4
1.3	Reading guide	4
2	Platform Architecture	5
2.1	The Rust/WASM core	5
2.2	The TypeScript monorepo and the <code>wpm</code> CLI	5
2.3	Supporting crates	7
2.3.1	prolog8 — byte-capped proof engine	7
2.3.2	miniml-core — SIMD machine learning	7
2.3.3	ocpq — object-centric process querying	7
2.3.4	wasm4pm-cli and wasm4pm-cognition	7
3	The Cognition Periodic Table	8
3.1	Breeds as elements	8
3.2	The lawful lifecycle: <code>run_breed</code>	9
3.3	The ten-rung standing ladder	10
3.4	Contracts at the WASM boundary	11
4	The Anti-Cheat Oracle Calculus	12
4.1	Threat model	12
4.2	The universal laws U1–U6	12
4.3	52 oracles, 52 adversaries	12
4.4	Fresh-name hygiene	13
4.5	The integration suite	14
5	Paper-Grounded Falsification	15
5.1	Method: confirm and refute	15
5.2	Breeds, source papers, and distinguishing assertions	15

6	Process Truth: OCEL, Receipts, and the Release Certificate	18
6.1	OCPN lifecycle models	18
6.2	Measured-fitness reports	18
6.3	The BLAKE3 receipt chain	19
6.4	The release certificate	19
7	Quality Evidence	20
7.1	Test suites at v26.6.10	20
7.2	Defense in depth	20
8	Conclusion	22
A	Full Breed Roster	23

Chapter 1

Introduction

1.1 What wasm4pm is

wasm4pm is a process-mining platform. Its purpose is the classical process-mining triad — *discovery* of process models from event logs, *conformance* checking of logs against models, and *enhancement* of models with performance and predictive information [1] — delivered as a deterministic WebAssembly core that runs identically in Node.js, browsers, and embedded hosts.

The repository at release v26.6.10 contains two principal layers and a family of supporting crates:

1. **The Rust/WASM core** (`wasm4pm/`). Sixty process-mining algorithms compiled via `wasm-pack`, exported through `wasm-bindgen`. The API surface, documented in `WASM_API.md`, comprises 335 public Rust→JavaScript exports across more than one hundred source modules, spanning core discovery (DFG, Alpha++, Heuristic, Inductive miners; metaheuristic miners using ACO, PSO, genetic search, A*, hill climbing, simulated annealing, ILP; DECLARE; streaming variants), conformance and alignment, POWL partial-order workflow handling, temporal-profile analysis, social networks, simulation, ML algorithms, and autonomic control loops.
2. **The TypeScript monorepo** (`packages/` and `apps/`). Twelve workspace packages — `agents`, `cognition`, `config`, `contracts`, `engine`, `kernel`, `ml`, `observability`, `planner`, `supabase`, `testing`, plus the applications — with the published CLI, `wpm`, shipping from `apps/wasm4pm/` as `@wasm4pm/cli`. A web playground lives at `apps/playground-web/`. The workspace version at the time of writing is `26.6.10` (CalVer `vYY.M.D`).
3. **Supporting crates** (`crates/`): `prolog8`, a byte-capped deterministic proof engine; `miniml-core`, a SIMD-accelerated machine-learning core; `ocpq`, object-centric process querying; `wasm4pm-cli`, a development-only Rust CLI; and `wasm4pm-cognition`, the cognition layer that is the subject of Chapters 3–5.

1.2 The doctrine: process truth

The platform’s constitution is borrowed from van der Aalst’s program of evidence-based process science [1, 2]:

*If the code says it worked but the event log cannot prove a lawful process happened,
then it did not work.*

This is not a slogan; it is implemented as a runtime gate. Every cognition breed execution derives an object-centric event log (OCEL) from its own inference trace and replays that log against a declared lifecycle model; non-conformance (fitness < 1.0) converts an apparently successful run into a hard error (Section 3.2). Every `wpm` run emits a BLAKE3 receipt with non-empty input and output hashes. Releases are certified by a machine-checked certificate (Chapter 6).

1.3 Reading guide

Chapter 2 describes the platform architecture. Chapter 3 presents the cognition periodic table: 52 `PARTIAL_ALIVE` breeds, the lawful `run_breed` lifecycle, and the ten-rung standing ladder. Chapter 4 develops the anti-cheat oracle calculus U1–U6 with its paired adversaries. Chapter 5 documents paper-grounded falsification: every breed is tested against a fixture derived from its source publication, and the oracle must both *confirm* the published behavior and *refute* a deliberately corrupted variant. Chapter 6 covers the process-truth machinery — OCPN lifecycle models, measured-fitness reports, receipts, and the release certificate. Chapter 7 summarizes the quality evidence, and Chapter 8 concludes.

Chapter 2

Platform Architecture

2.1 The Rust/WASM core

The mining core lives in `wasm4pm/` and compiles to a single WASM binary via `wasm-pack build -target nodejs`. Its design principles, stated in `WASM_API.md`, treat the WASM boundary as a *sovereign sanctuary*:

1. **Memory isolation.** Every call executes within linear memory, isolated from host non-determinism.
2. **Deterministic calculus.** Every algorithm must satisfy $f(BinaryHash, InputHash, Seed) \rightarrow (Output, Receipt)$, with bit-exact identity required for Rank-1 oracles.
3. **Immutable verdicts.** A `Refused` verdict from the kernel is process law; no administrative layer can overturn it.

The exported module families (counts as documented and verified in `WASM_API.md` at the current tree) are summarized in Table 2.1.

Two deliberately documented API quirks illustrate the project’s verification culture: `discover_astar` returns `iterations_used` (an iteration count, explicitly *not* a fitness score), and the metaheuristic miners (`discover_aco_algorithm`, `discover_pso_algorithm`) return a typed `Err("no_edges")` rather than fabricating a model when the log has no directly-follows edge vocabulary. Refusal is a first-class outcome throughout the system.

2.2 The TypeScript monorepo and the wpm CLI

The monorepo (pnpm workspaces, version 26.6.10) layers typed contracts and orchestration over the WASM core:

- `packages/kernel` — the WASM loader and execution kernel; the published npm tarball `wasm4pm-26.6.10.tgz` is smoke-tested from `packages/kernel/` during release certification.

Table 2.1: WASM core module families (from `WASM_API.md`, 335 total exports).

Module family	Exports	Purpose
Core Discovery	15	DFG, Alpha++, Heuristic, Inductive, ACO, PSO, genetic, A*, hill climbing, simulated annealing, ILP, DECLARE, streaming variants
ML Algorithms	6+	classification, clustering, forecasting, anomaly detection, regression, PCA
Streaming	20+	real-time DFG, SIMD acceleration, conformance
POWL	8+	partial-order workflow parse/simplify/convert
Analysis	20+	conformance, simulation, performance, temporal, social networks
Prolog8	3	query evaluation, proof verification, capabilities
TrueX	1	OCEL 2.0 execution-receipt verification (BLAKE3 + JCS-OCEL canonicalization)
Autonomic	10+	RL orchestrator, SPC, circuit breaker, self-healing
AutoMembrane	25+	motion classification, actor/object/route/ML/time envelopes, benchmarking

- `packages/engine`, `packages/contracts`, `packages/config` — algorithm orchestration, typed input/output contracts, and configuration (environment prefix `WASM4PM_*`; exit codes 0 ok, 1 config, 2 source, 3 exec, 4 partial, 5 system).
- `packages/cognition` — the TypeScript face of the cognition layer; field contracts mirror the Rust WASM output exactly (e.g. `cognition_run` returns `{status, breed, run_id, output_hash, replay_pointer, options_profile, output}`).
- `packages/observability` — OpenTelemetry span emission; project rule: every public operation emits spans with `service_name` and a `status` of "ok" or "error", and no success claim is accepted without span evidence.
- `packages/agents`, `packages/planner`, `packages/ml`, `packages/supabase`, `packages/testing` — agent harnesses, planning, ML bindings, persistence, and the shared test harness.

The published CLI, `wpm` (`apps/wasm4pm/`, npm package `@wasm4pm/cli`), exposes mining and cognition commands, including `wpm run`, `wpm cognition run`, and `wpm compile -spec <f.json> [-run]`, a multi-stage compilation pipeline in which an unknown breed exits with code 2. Every `wpm run` and `wpm cognition run` writes a BLAKE3 receipt to `.wasm4pm/receipts/latest.json`; there is deliberately no `-no-receipt` escape hatch.

2.3 Supporting crates

2.3.1 prolog8 — byte-capped proof engine

`crates/prolog8` is a compact, deterministic inference engine that evaluates queries against fact blocks and Horn rules, emitting typed proof DAGs with BLAKE3 receipt chains. It is designed for bounded-memory embedded use: hard byte caps (e.g. a maximum predicate arity of 8) make worst-case resource consumption a static property. Its use cases inside the platform are proof-of-admissibility gates in autonomic subsystems and receipt-chain validation in the manufacturing-style pipelines. Source modules: `admission.rs`, `catalog.rs`, `hash.rs`, `kernel.rs`, `replay.rs`, `types.rs`, `wasm.rs`.

2.3.2 miniml-core — SIMD machine learning

`crates/miniml-core` (published as `wminml`) is a minimal machine-learning core: 70+ algorithms across 15 families, SIMD-accelerated matrix operations using `v128` intrinsics, zero-copy serde serialization, and AutoML via genetic-algorithm feature selection with PSO hyperparameter optimization. It backs the ML exports of the WASM core.

2.3.3 ocpq — object-centric process querying

`crates/ocpq` implements object-centric process querying in the tradition of Küsters and van der Aalst’s OCPQ line of work [3]: binding-based queries over OCEL 2.0 logs in which constraints range over events *and* objects simultaneously, rather than flattening the log to a single case notion. The crate ships with its own test and benchmark suites (`crates/ocpq/tests`, `crates/ocpq/benches`).

2.3.4 wasm4pm-cli and wasm4pm-cognition

`crates/wasm4pm-cli` is a development-only Rust CLI (the shipped CLI is the TypeScript `wpm`). `crates/wasm4pm-cognition` is the cognition layer, treated at length in the remainder of this monograph; it builds both natively and for `wasm32-unknown-unknown` (the `wasm32` gate is a required check), and exposes `cognition_run`, `cognition_verify`, and `system_build` through `wasm-bindgen`.

Chapter 3

The Cognition Periodic Table

3.1 Breeds as elements

The cognition layer organizes classical symbolic-AI reasoning systems as *breeds* — each a faithful, deterministic reconstruction of one historical reasoning paradigm, normalized behind one trait (`CognitionBreed`) and one WASM export (`cognition_run`). The metaphor is a periodic table: each breed is an element with a fixed identity, a family, a historical ancestor, and a measured standing.

The registry, `crates/wasm4pm-cognition/breeds/registry.json`, contains **55 entries** at v26.6.10, of which **52 carry status `partial_alive`** and 3 remain `UNSUPPORTED`. Each entry binds a breed to its schemas, oracle suite, OCEL model, and receipt schema. A representative entry:

```
{
  "breed_id": "mycin",
  "breed_name": "MYCIN",
  "historical_ancestor": "MYCIN (Shortliffe, 1976)",
  "generalized_family": "rule_based_cf",
  "input_schema": "mycin_input_v1",
  "output_schema": "mycin_output_v1",
  "specification_relation": "forward_chaining_cf",
  "oracle_suite_id": "oracle_mycin_v1",
  "ocel_model_id": "ocel_mycin_v1",
  "receipt_schema_id": "receipt_mycin_v1",
  "wasm_export_name": "cognition_run",
  "status": "PARTIAL_ALIVE",
  "standing": "DISPATCHABLE"
}
```

The standings recorded in the registry at v26.6.10 are: **36 dispatchable**, **16 traceable**, and **3 registered** (the latter being the three `UNSUPPORTED` entries). The breed source tree, `src/breeds/`, holds the 52 implementations plus dispatch, standing, and support modules.

The families span the breadth of symbolic AI: rule-based reasoning with certainty factors (MYCIN), heuristic search and means-ends analysis (GPS, STRIPS), logic programming and

its abductive, inductive, constraint, and probabilistic extensions (Prolog, abductive LP, ILP, CLP, ProbLog), answer set programming, non-monotonic logics (default logic, circumscription), description logic ($\mathcal{EL}$ subsumption), temporal reasoning (Allen’s interval algebra, event calculus, situation calculus, LTL monitoring, CTL model checking), uncertainty calculi (Bayesian networks, Dempster–Shafer, fuzzy logic, Markov logic), planning (HTN, partial-order, contingent, MDP, POMDP), learning (version spaces, EBL, Q-learning, case-based reasoning), cognitive architectures (Soar, ACT-R, episodic memory), language and knowledge structures (ELIZA, conceptual-dependency scripts, construction grammar, frames), qualitative and naive physics, analogy (SME), blackboard architectures (Hearsay-II), SAT solving (CDCL), tableaux proof, belief merging, and meta-reasoning.

3.2 The lawful lifecycle: `run_breed`

No `breed` is invoked directly. All execution flows through `run_breed` in `src/breeds/dispatch.rs`, which enforces a five-stage lawful lifecycle:

1. **Preconditions.** `b.preconditions(input)` validates the contract and the breed’s declared complexity caps; failure is a refusal, not a degraded run.
2. **Run.** `b.run(input)` executes the algorithm.
3. **FM-5 gate.** A framework-level fraud gate, independent of the breed’s own checks: an empty `inference_trace` on a “successful” run is rejected as an FM-5 fraud signal. A breed cannot claim success without showing its work.
4. **Postconditions.** `b.postconditions(input, &output)` verifies output-side invariants.
5. **OCEL derivation and conformance.** The inference trace is converted into an object-centric event log (`derive_ocel`); if a lifecycle model is declared for the breed, the log is replayed against it (`validate_ocel_alignment`) and any non-conformance aborts the run with the measured fitness in the error message.

The decisive excerpt (verbatim from `dispatch.rs`):

```
pub fn run_breed(b: &dyn CognitionBreed, input: &BreedInput)
    -> Result<BreedOutput, String>
{
    b.preconditions(input)
        .map_err(|e| format!("{}", precondition failed: {}, b.id(), e))?;
    let mut output = b.run(input)
        .map_err(|e| format!("{}", {}, e.breed, e.message))?;
    // Framework-level FM-5 gate: a breed cannot claim success without an
    // inference trace, independent of its own postconditions.
    if output.inference_trace.is_empty() {
        return Err(format!(
            "{}: empty inference trace (FM-5 fraud signal, framework gate)",
            b.id()));
    }
}
```

```

}
b.postconditions(input, &output)
  .map_err(|e| format!("{: postcondition failed: {}", b.id(), e))?;

// Derive OCEL and validate conformance (van der Aalst doctrine)
let breed_id = format!("{}", b.id());
let trace_str =
  serde_json::to_string(&output.inference_trace).unwrap_or_default();
let tmp_run_id =
  blake3::hash(trace_str.as_bytes()).to_hex().to_string();
let ocel_log =
  crate::ocel::derive_ocel(&breed_id, &tmp_run_id,
    &output.inference_trace);
if let Some(model) = crate::ocel::lifecycle_model_for(&breed_id) {
  let conformance =
    crate::ocel::validate_ocel_alignment(&ocel_log, model);
  if !conformance.is_conforming {
    return Err(format!(
      "{: OCEL conformance failure (fitness={:.3}): {}",
      breed_id, conformance.fitness,
      conformance.refusals.join("; ")));
  }
}
output.ocel_log =
  Some(serde_json::to_value(&ocel_log).unwrap_or(
    serde_json::Value::Null));
Ok(output)
}

```

Dispatch itself (`dispatch_breed`) parses the breed string into the `BreedId` enum and matches *exhaustively*: the Rust compiler enforces that every named breed has a dispatch arm, so a breed cannot be registered yet silently unreachable.

Determinism is structural: all breed collections are `BTreeMap`/`BTreeSet` or sorted vectors (hash-map iteration order would break determinism and receipts); the only permitted RNG is the seeded `support::rng::seeded_rng()`; and a shared `assert_deterministic` harness compares full `BreedOutput` bytes across repeated runs (hand-rolled determinism checks are themselves classified as a cheat, code A11).

3.3 The ten-rung standing ladder

A breed's maturity is not a boolean. `src/breeds/standing.rs` defines `BreedStanding`, a strictly ordered ten-rung ladder; the documentation comments on the enum variants are reproduced in Table 3.1.

The constant `BreedStanding::PARTIAL_ALIVE_MINIMUM` is `DISPATCHABLE`: a registry entry may flip from `UNSUPPORTED` to `PARTIAL_ALIVE` only after the build and test gates are green *and* an OCEL report with measured-fitness provenance exists for the breed. At v26.6.10 the population sits at rungs 3–6: 36 breeds at `DISPATCHABLE`, 16 at `TRACEABLE`, 3 (unsupported) at

Table 3.1: The `BreedStanding` ladder (`src/breeds/standing.rs`).

Rung	Standing	Requirement
1	NAMED	<code>BreedId</code> variant exists in the enum and <code>BreedId::ALL</code> .
2	REGISTERED	Entry present in <code>breeds/registry.json</code> with mandatory fields.
3	DISPATCHABLE	Dispatch arm exists in <code>dispatch_breed_id</code> (compile-enforced). Minimum standing for <code>PARTIAL_ALIVE</code> status.
4	BOUNDED	<code>complexity_caps</code> declared; <code>preconditions()</code> enforces them.
5	ORACLED	Oracle fixture exists; paper <code>expected_value</code> asserted.
6	TRACEABLE	<code>inference_trace</code> non-empty; trace-law checks pass.
7	CANONICAL	BLAKE3 receipt deterministic; <code>SortedFacts</code> used throughout.
8	REFUSABLE	<code>CognitionError</code> variants in use; all refusal paths tested.
9	REPLAYABLE	OCEL conformance fitness = 1.0; <code>replay_pointer</code> valid.
10	CERTIFIED	Ed25519 signature verified; Merkle root anchored; counter-test green.

REGISTERED. The ladder makes “how alive is this breed?” a queryable, monotone fact rather than a vibe.

3.4 Contracts at the WASM boundary

The cognition layer’s WASM surface (`src/wasm.rs`) is contract-exact and mirrored verbatim into the TypeScript layer:

- `cognition_run` input: `{ breed, contract, options?: { profile? } }` — `serde deny_unknown_fields` rejects bare breed inputs.
- `cognition_run` output (`ContractResult`): `{ status: "ok", breed, run_id, output_hash, replay_pointer, options_profile, output }`, where `replay_pointer` is the first 16 hex characters of `output_hash`.
- `cognition_verify` output (`VerifyResult`): `{ findings, status: "verified" | "has_findings" }`.
- `system_build` output: `{ pareto_front, dominated }` — breed-system composition as multi-objective selection.

Repository hooks (`cognition-contract-guard.sh`, `cognition-wasm-gate.sh`) enforce that no TypeScript consumer invents fields the Rust side does not emit.

Chapter 4

The Anti-Cheat Oracle Calculus

4.1 Threat model

The cognition layer assumes its own implementations are adversarial. The threat model (`docs/breeds/anti-cheat-threat-model.md`) catalogs the ways a breed could *appear* to implement a classical algorithm while actually doing something cheaper: hard-coding answers for known fixtures, emitting hollow traces, ignoring its input, never refusing malformed input, or faking determinism. Each cheat mode carries an **AC-*** taxonomy code (A1–A11), and each breed is required to have a *counter-test* that would catch its most tempting cheat. A representative entry: for `csp_ac3`, the feared cheat is “a backtracking-only solver that never does arc revision (gets right answers, fakes `csp-revise` steps)”; the counter-test asserts that the $K_3/2$ -coloring UNSAT instance is refuted by AC-3 *alone* — the trace must show a domain-wipeout `csp-revise` step and *zero* `csp-assign` steps, cross-checked by a CLP instance that must solve with zero backtracks.

4.2 The universal laws U1–U6

`src/breeds/support/oracle.rs` defines the universal anti-cheat calculus. `run_universal_anticheat` executes, in order, U1, U2, U2b, U3, U4, U5 and returns exactly six results; all must pass for a breed to be *oracle-green*. The mapping from law to cheat mode, taken from the module documentation, is given in Table 4.1.

If U1 fails, U2 and U2b are recorded as failed with the explicit detail “skipped because U1 failed” — skipping is never silent.

4.3 52 oracles, 52 adversaries

Each of the 52 breeds has a per-breed oracle implementation under `src/breeds/support/oracle_impls/` (organized by family: `dialogue.rs`, `learning.rs`, `logic.rs`, `planning.rs`, `rule_fact.rs`). Each oracle supplies the novel input, intermediate-step assertions, boundary pair, and refusal input for its breed — and, crucially, each oracle is paired with an *adversary*: a deliberately

Table 4.1: Universal anti-cheat laws (`support/oracle.rs`).

Law	Targets	Check
U1	A1, A2	A <i>novel</i> input (not present in any fixture) runs without error — defeats fixture memorization.
U2	A4	<code>assert_intermediate</code> passes on the novel trace — the trace must contain the algorithm’s real intermediate steps.
U2b	A4	<code>assert_trace_values</code> passes (Surface-3 content check) — step <i>values</i> , not just step kinds, must be meaningful.
U3	A3, A6	A boundary pair of inputs must yield different outputs — defeats constant-output and input-insensitive breeds.
U4	A5	A designated refusal input must return <code>Err</code> — a breed that never refuses is cheating.
U5	A11	Determinism over full <code>BreedOutput</code> bytes — defeats sham determinism.
U6	meta	Adversary check: a known-cheating implementation must <i>fail</i> the oracle — the oracle itself is tested.

cheating implementation carrying its `AC-*` code, used by U6 to verify the oracle has discriminating power. The framework ships a generic hollow-trace adversary (cheat code `AC-A4`) that emits a trace of meaningless kinds; the meta-oracle asserts not merely that the adversary fails but that the failure detail *names the cheat code*:

```
assert!(detail.contains("AC-A4"),
    "detail must name cheat code: {}", detail);
```

This closes the classic self-reference loophole (FM-5): a test suite that only ever runs honest implementations can never demonstrate that it would catch a dishonest one. Here, for every breed, the suite demonstrably rejects a known cheat.

4.4 Fresh-name hygiene

Hidden-oracle inputs use `uo_*`-prefixed fresh names (universal oracle namespace). A grep gate in `anti_fraud_gate.rs` enforces that no `uo_*` fresh name appears anywhere in `src/breeds/` — a breed implementation that special-cases the oracle’s secret inputs (cheat A8) is caught at test time by textual quarantine, independent of any behavioral check.

4.5 The integration suite

`tests/universal_anticheat.rs` contains **54 test functions** (verified by counting `#[test]` attributes), covering all 52 breeds plus framework-level meta-tests. All 54 pass at v26.6.10.

Chapter 5

Paper-Grounded Falsification

5.1 Method: confirm and refute

Every breed must be anchored to its source publication, not to the implementer’s intuition. `tests/paper_falsification.rs` iterates over the fixture directory `tests/fixtures/papers/` — **52 fixture files**, one per `PARTIAL_ALIVE` breed — and for each fixture performs both directions of a Popperian check:

1. **Confirm:** dispatch the fixture input through the full lawful path (`run_breed`, i.e. including the FM-5 gate and OCEL conformance) and assert the published expected value.
2. **Refute:** dispatch a corrupted variant and assert the oracle rejects it — a fixture that cannot fail is not evidence.

Each fixture carries a **provenance** block with a full bibliographic citation (and, where the project archive holds the PDF, a path to it), and paper-derived fixtures must assert the *published* number — for example, the Bayesian-network fixture asserts Pearl’s value 0.284171835 to full precision rather than a rounded approximation, and the ProbLog fixture asserts the success probability 0.552.

5.2 Breeds, source papers, and distinguishing assertions

Table 5.1 samples 18 representative breeds. Citations are transcribed from the on-disk fixture provenance blocks; the “distinguishing assertion” column states the fixture’s `expected.value` or the behavioral property the oracle pins down.

Table 5.1: Paper-grounded fixtures (sample of 18 of 52; tests/fixtures/papers/*.json).

Breed	Source paper (fixture provenance)	Distinguishing assertion
mycin	Shortliffe & Buchanan (1975), <i>A model of inexact reasoning in medicine</i> [4]	Certainty-factor accumulation across chained rules reproduces the CF-model combination values.
strips	Fikes & Nilsson (1971), <i>STRIPS: A new approach to the application of theorem proving to problem solving</i> [5]	Plan found via add/delete-list operator application on the published blocks domain.
gps	Newell & Simon (1963), <i>GPS, a program that simulates human thought</i> [6]	Means-ends analysis reduces the published difference table to a solved goal stack.
eliza	Weizenbaum (1966), <i>ELIZA — a computer program for the study of natural language communication between man and machine</i> [7]	Keyword-rank decomposition/reassembly reproduces script-driven responses.
bayesian_network	Pearl (1988), <i>Probabilistic Reasoning in Intelligent Systems</i> [8]	Posterior asserted to published precision: 0.284171835.
dempster_shafer	Shafer (1976), <i>A Mathematical Theory of Evidence</i> [9]	Combined belief 0.99 via Dempster’s rule on the fixture mass functions.
fuzzy_logic	Mamdani & Assilian (1975), <i>An experiment in linguistic synthesis with a fuzzy logic controller</i> [10]	Defuzzified output 41.66667 (centroid of clipped consequents).
default_logic	Reiter (1980), <i>A logic for default reasoning</i> [11]	Extension computation on the canonical default theory.
circumscription	McCarthy (1980), <i>Circumscription — a form of non-monotonic reasoning</i> [12]	Minimal-model result: <code>flies_tweety=true</code> , <code>flies_opus=false</code> .
allen_temporal	Allen (1983), <i>Maintaining knowledge about temporal intervals</i> [13]	Path-consistency over the 13 interval relations yields <code>temporal-consistent</code> .
csp_ac3	Mackworth (1977), <i>Consistency in networks of relations</i> [14]	$K_3/2$ -coloring refuted by arc revision alone: a <code>csp-revise</code> domain wipeout and zero <code>csp-assign</code> steps.
asp	Gelfond & Lifschitz (1988), <i>The stable model semantics for logic programming</i> [15]	Stable models of the published program enumerated exactly.
clp	Jaffar & Lassez (1987), <i>Constraint logic programming</i> [16]	Solution <code>x=6</code> , <code>y=3</code> by propagation (zero backtracks on the pure-propagation instance).

Breed	Source paper (fixture provenance)	Distinguishing assertion
version_space	Mitchell (1982), <i>Generalization as search</i> [17]	S/G boundary-set convergence on the published example sequence.
rl_symbolic	Watkins & Dayan (1992), <i>Q-learning</i> [18]	Q-value convergence verified on the fixture MDP.
analogy_sme	Falkenhainer, Forbus & Gentner (1989), <i>The Structure-Mapping Engine</i> [19]	Mapping sun → nucleus , planet → electron for the solar-system/atom analogy.
sat_cdcl	Marques-Silva & Sakallah (1999), <i>GRASP: a search algorithm for propositional satisfiability</i> [20]	UNSAT verdict with conflict-driven clause learning on the fixture formula.
problog	De Raedt, Kimmig & Toivonen (2007), <i>ProbLog: a probabilistic Prolog and its application in link discovery</i> [21]	Success probability 0.552000 on the fixture program.

The remaining 34 fixtures follow the same pattern, citing inter alia Anderson & Lebiere (ACT-R), Laird, Newell & Rosenbloom (Soar) [22], Kowalski (predicate logic as programming language) [23], Kakas, Kowalski & Toni (abductive LP), Quinlan (FOIL), Minsky (frames) [24], Schank & Abelson (scripts), Erman et al. (Hearsay-II), Feigenbaum, Buchanan & Lederberg (DENDRAL), Aamodt & Plaza (CBR), Clarke, Emerson & Sistla (CTL), Havelund & Rosu (LTL monitoring), Nau et al. (SHOP2 HTN), McAllester & Rosenblitt (systematic nonlinear planning), Kaelbling, Littman & Cassandra (POMDP), Bellman (dynamic programming), Richardson & Domingos (Markov logic), Konieczny & Pino Pérez (belief merging), Baader, Brandt & Lutz ($\mathcal{EL}$), Smullyan (tableaux), de Kleer & Brown (qualitative physics), Hayes (naive physics), Tulving and Nuxoll & Laird (episodic memory), Goldberg (construction grammar), Marr & Poggio (stereo disparity), Sussman (HACKER), and Cox & Raja (metareasoning). Several fixtures are marked **citation-only** where no PDF is archived; in those cases the citation alone is the provenance and no quoted text is asserted.

Chapter 6

Process Truth: OCEL, Receipts, and the Release Certificate

6.1 OCPN lifecycle models

Each breed declares an L1 lifecycle model as an object-centric Petri net in `ocel/models/l1/<breed_id>.ocpn.json` — **52 models**, one per `PARTIAL_ALIVE` breed (verified by directory count). The model names the lawful step kinds and their ordering; for MYCIN, for example, the declared steps are `rule_selected`, `premise_matched`, `cf_accumulated`, `threshold_checked`, `diagnosis_emitted`.

6.2 Measured-fitness reports

`ocel/reports/` holds **52 fitness reports**, one per breed. Each records the model ID, model level, report date, method, and the measured fitness with provenance. The MYCIN report (verbatim fields):

```
{
  "breed_id": "mycin",
  "model_id": "mycin-l1-v1",
  "model_level": "L1",
  "report_date": "2026-06-10",
  "method": "native_dfa_replay",
  "fitness": 1,
  "fitness_note": "Alignment fitness = 1.0 measured by
    validate_ocel_alignment() (native Rust DFA replay).
    Conformance gate enforced in run_breed()".
}
```

Project law: fitness above 0.85 is required for a model to be considered valid at all, and route admission into the broader MCPP ecosystem requires fitness of exactly 1.0. Because the conformance check runs *inside* `run_breed`, the reports are not aspirational documentation — a breed whose traces stopped conforming would start failing at runtime.

6.3 The BLAKE3 receipt chain

Every `wpm run` and `wpm cognition run` emits a receipt to `.wasm4pm/receipts/latest.json` with non-empty `input_hash` and `output_hash` (BLAKE3). On the Rust side, the `run_id` is itself a BLAKE3 hash, and `replay_pointer` (the first 16 hex characters of `output_hash`) lets any consumer re-locate and re-verify the exact output later. The TrueX module in the WASM core verifies OCEL 2.0 execution receipts using BLAKE3 over a JCS-canonicalized OCEL profile, so receipt verification is available inside the sandbox, not only in the host.

6.4 The release certificate

`RELEASE_CERTIFICATE.v26.6.10.json` is the machine-checked release artifact for v26.6.10. Its verified contents are summarized in Table 6.1.

Table 6.1: Release certificate v26.6.10 (`RELEASE_CERTIFICATE.v26.6.10.json`).

Field	Value
Package	<code>wasm4pm 26.6.10</code> , git commit <code>eceacfc43d5...</code>
Reachability	60 of 60 algorithms reachable; reachability hash <code>92e0264e...</code>
Behavior	60 algorithms $\times$ (60 positive, 120 negative, 60 invariant) cases; <code>all_failed_correctly: true</code> ; evidence hash <code>b79d5e7f...</code>
Examples	8 examples, 64 total executions; manifest hash <code>86933a9b...</code>
Artifact	<code>wasm4pm-26.6.10.tgz</code> ; sha512 integrity <code>A6g+g0qY...</code> ; pack-smoke path <code>packages/kernel/</code> ; WASM bundle hash <code>d4bc2f40...</code>

Three properties deserve emphasis. First, *reachability* is asserted per algorithm: all 60 mining algorithms are demonstrably callable through the published package, not merely compiled in. Second, the behavior evidence is dominated by *negative* cases — 120 negative against 60 positive — and the certificate’s `all_failed_correctly: true` asserts that every injected invalid input was refused with the correct error, in line with the negative-testing requirement of the process-truth constitution. Third, the certificate pins the exact tarball and WASM binary by hash, closing the gap between “the tests passed” and “the artifact you installed is the artifact that passed.”

Chapter 7

Quality Evidence

7.1 Test suites at v26.6.10

The evidence base, with each figure verified against the working tree:

- **Universal anti-cheat:** 54 test functions in `tests/universal_anticheat.rs` (counted directly), all green — every breed passes U1–U5 + U2b, and every paired adversary is rejected by U6 with its cheat code named.
- **Paper falsification:** `tests/paper_falsification.rs` drives all 52 paper fixtures (counted directly in `tests/fixtures/papers/`) through the confirm-and-refute protocol via the full lawful dispatch path.
- **Cargo suites:** the cognition crate’s unit, integration, and anti-fraud suites are green at the release commit; the v26.6.10 integration commit records 868 cargo tests and 237 vitest tests passing across the workspace.
- **wasm32 gate:** `cargo check -target wasm32-unknown-unknown -features wasm` is a required release gate; the cognition layer builds for the WASM target, and the WASM bundle hash is pinned in the release certificate.
- **TypeScript:** the monorepo builds and tests under pnpm at workspace version 26.6.10; the FM-5 rule requires at least one cognition integration test to exercise the *real* WASM binary with no mocking of `init.js`.
- **Release certification:** 60/60 reachability, 60/120/60 behavior cases with `all_failed_correctly`, tarball and bundle integrity hashes (Table 6.1).
- **OCEL conformance:** 52 measured-fitness reports, each recording fitness 1.0 by native DFA replay, with the same check enforced at runtime inside `run_breed`.

7.2 Defense in depth

The quality argument is layered so that no single artifact has to be trusted:

1. The *compiler* enforces dispatch totality (exhaustive enum match) and collection determinism (BTree types).
2. The *framework* enforces the FM-5 empty-trace gate and OCEL conformance on every run, in production code, not only in tests.
3. The *oracles* enforce behavioral fidelity (U1–U5), and the *adversaries* enforce oracle strength (U6).
4. The *papers* anchor expected values to published numbers with bibliographic provenance, and the refute direction proves the fixtures can fail.
5. The *receipts and certificate* anchor the shipped artifact to the evidence by cryptographic hash.

A failure anywhere in this stack is, by project law, an Andon event: the line stops on `error[E, test.*FAILED, FM-5 violation, or panicked at`, and the defect is fixed before any other work proceeds.

Chapter 8

Conclusion

wasm4pm at v26.6.10 demonstrates that a process-mining platform can hold itself to the standard it applies to the processes it mines. The mining core supplies 60 algorithms behind a deterministic WASM boundary; the cognition layer revives 52 classical reasoning systems as breeds, each paper-anchored, oracle-guarded, adversary-tested, and conformance-gated at runtime; and the release machinery converts “it works” from an assertion into a certificate. The discipline throughout is van der Aalst’s, applied reflexively: the system does not ask to be believed — it asks to be replayed.

Future work tracked in the repository includes advancing breeds up the standing ladder toward `REPLAYABLE` and `CERTIFIED` (Ed25519-signed, Merkle-anchored), promoting the three remaining `REGISTERED` entries, and validating the mining algorithms against external real-world XES/OCEL corpora.

Appendix A

Full Breed Roster

Table A.1 reproduces the complete registry (`crates/wasm4pm-cognition/breeds/registry.json`, 55 entries) at v26.6.10, sorted by breed identifier. The `historical_ancestor` and `generalized_family` columns are transcribed verbatim, including the literal placeholder string TBD where the registry has not yet been enriched for an entry — these placeholders are metadata-completeness debt, not implementation debt; every DISPATCHABLE and TRACEABLE breed below has a full implementation, oracle, adversary, paper fixture, OCPN model, and fitness report. The three REGISTERED entries (`morphological`, `ocpm_route_discoverer`, `triz`) are the UNSUPPORTED breeds awaiting promotion.

Table A.1: Complete breed registry at v26.6.10 (55 entries; 36 DISPATCHABLE, 16 TRACEABLE, 3 REGISTERED).

Breed	Historical ancestor	Family	Standing
<code>abductive_ibe</code>	TBD	TBD	TRACEABLE
<code>abductive_lp</code>	Abduction (Peirce, 1878) (abductive)	<code>R_generalized_abductive</code>	TRACEABLE
<code>act_r</code>	TBD	TBD	TRACEABLE
<code>allen_temporal</code>	Temporal Logic (Allen, 1983) (temporal)	<code>R_generalized_temporal</code>	DISPATCHABLE
<code>analogy_sme</code>	Structure Mapping (Gentner, 1983) (analogical)	<code>R_generalized_analogical</code>	DISPATCHABLE
<code>asp</code>	TBD	TBD	TRACEABLE
<code>autoinstinct_learning</code>	<code>autoinstinct_learning</code>	<code>autoinstinct</code>	DISPATCHABLE
<code>autoinstinct_neurosis</code>	<code>autoinstinct_neurosis</code>	<code>autoinstinct</code>	DISPATCHABLE
<code>autoinstinct_semantics</code>	<code>autoinstinct_semantics</code>	<code>autoinstinct</code>	DISPATCHABLE
<code>autoinstinct_vision</code>	<code>autoinstinct_vision</code>	<code>autoinstinct</code>	DISPATCHABLE
<code>bayesian_network</code>	Bayesian Networks (Pearl, 1985) (bayesian)	<code>R_generalized_probabilistic</code>	TRACEABLE
<code>belief_merging</code>	TBD	TBD	TRACEABLE
<code>cbr</code>	CBR (Kolodner, 1983)	<code>case_based_reasoning</code>	DISPATCHABLE
<code>circumscription</code>	TBD	TBD	TRACEABLE
<code>clp</code>	TBD	TBD	TRACEABLE

Breed	Historical ancestor	Family	Standing
construction_grammar	Construction Grammar (Goldberg, 1995)	grammar_semantics	DISPATCHABLE
contingent_plan	AND-OR belief-state search (Russell & Norvig, AIMA §4.3.2)	planning_under_uncertainty	DISPATCHABLE
csp_ac3	CSP (Mackworth, 1977) (constraint)	R_generalized_constraint	TRACEABLE
ctl_check	TBD	TBD	DISPATCHABLE
default_logic	TBD	TBD	DISPATCHABLE
dempster_shafer	Dempster-Shafer (Shafer, 1976) (dempster_shafer)	R_generalized_evidential	DISPATCHABLE
dendral	DENDRAL (Feigenbaum et al., 1969)	hypothesis_enumeration	DISPATCHABLE
description_logic	Description Logics (Baader et al., 1991) (ontological)	R_generalized_ontological	DISPATCHABLE
ebl	TBD	TBD	DISPATCHABLE
eliza	ELIZA (Weizenbaum, 1966)	pattern_matching_dialogue	DISPATCHABLE
episodic_memory	TBD	TBD	TRACEABLE
event_calculus	TBD	TBD	DISPATCHABLE
frames_inheritance	TBD	TBD	DISPATCHABLE
fuzzy_logic	Fuzzy Sets (Zadeh, 1965) (fuzzy)	R_generalized_fuzzy	DISPATCHABLE
gps	GPS (Newell & Simon, 1957)	means_ends_analysis	DISPATCHABLE
hearsay	Hearsay-II (Erman et al., 1980)	blackboard_architecture	DISPATCHABLE
htn_planning	SHOP2 (Nau et al., 2003)	hierarchical_task_network	DISPATCHABLE
ilp	ILP (Muggleton, 1991) (inductive)	R_generalized_inductive	DISPATCHABLE
ltl_monitor	LTL Monitor (Havelund & Roşu, 2001) (ltl_monitor)	R_generalized_temporal	TRACEABLE
markov_logic	Markov logic networks (Richardson & Domingos, 2006)	statistical_relational	DISPATCHABLE
mdp	TBD	TBD	TRACEABLE
meta_reasoning	Metareasoning (Cox & Raja, 2011)	meta_level_control	DISPATCHABLE
morphological	TBD	TBD	REGISTERED
mycin	MYCIN (Shortliffe, 1976)	rule_based_cf	DISPATCHABLE
naive_physics	TBD	TBD	DISPATCHABLE
ocpm_route_discoverer	TBD	TBD	REGISTERED
partial_order_plan	TBD	TBD	DISPATCHABLE
pomdp	POMDP planning (Kaelbling, Littman & Cassandra, 1998)	decision_theoretic	TRACEABLE
problog	TBD	TBD	TRACEABLE
prolog	Prolog (Colmerauer & Roussel, 1972)	logic_programming	DISPATCHABLE
qualitative_reason	TBD	TBD	DISPATCHABLE
rl_symbolic	TBD	TBD	DISPATCHABLE
sat_cdcl	TBD	TBD	TRACEABLE
script_sam	TBD	TBD	DISPATCHABLE
situation_calculus	TBD	TBD	DISPATCHABLE
soar	SOAR (Laird, Newell & Rosenbloom, 1987)	unified_cognitive_architecture	DISPATCHABLE
strips	STRIPS (Fikes & Nilsson, 1971)	classical_planning	DISPATCHABLE

Breed	Historical ancestor	Family	Standing
tableaux	Analytic tableaux (Smullyan, 1968)	proof_search	DISPATCHABLE
triz	TBD	TBD	REGISTERED
version_space	TBD	TBD	TRACEABLE

Bibliography

- [1] W. M. P. van der Aalst. *Process Mining: Data Science in Action*, 2nd edition. Springer, 2016.
- [2] W. M. P. van der Aalst. *Object-Centric Process Mining: Dealing with Divergence and Convergence in Event Data*. In: Software Engineering and Formal Methods (SEFM), 2019.
- [3] J. N. Küsters and W. M. P. van der Aalst. *OCPQ: Object-Centric Process Querying*. (Object-centric process querying line of work; basis of the `ocpq` crate.)
- [4] E. H. Shortliffe and B. G. Buchanan. *A model of inexact reasoning in medicine*. Mathematical Biosciences, 23(3–4), 1975.
- [5] R. E. Fikes and N. J. Nilsson. *STRIPS: A new approach to the application of theorem proving to problem solving*. Artificial Intelligence, 2(3–4), 1971.
- [6] A. Newell and H. A. Simon. *GPS, a program that simulates human thought*. In: Computers and Thought, 1963.
- [7] J. Weizenbaum. *ELIZA — a computer program for the study of natural language communication between man and machine*. Communications of the ACM, 9(1), 1966.
- [8] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.
- [9] G. Shafer. *A Mathematical Theory of Evidence*. Princeton University Press, 1976.
- [10] E. H. Mamdani and S. Assilian. *An experiment in linguistic synthesis with a fuzzy logic controller*. International Journal of Man-Machine Studies, 7(1), 1975.
- [11] R. Reiter. *A logic for default reasoning*. Artificial Intelligence, 13(1–2), 1980.
- [12] J. McCarthy. *Circumscription — a form of non-monotonic reasoning*. Artificial Intelligence, 13(1–2), 1980.
- [13] J. F. Allen. *Maintaining knowledge about temporal intervals*. Communications of the ACM, 26(11), 1983.
- [14] A. K. Mackworth. *Consistency in networks of relations*. Artificial Intelligence, 8(1), 1977.

- [15] M. Gelfond and V. Lifschitz. *The stable model semantics for logic programming*. In: Proceedings of the Fifth International Conference and Symposium on Logic Programming, 1988.
- [16] J. Jaffar and J.-L. Lassez. *Constraint logic programming*. In: Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL), 1987.
- [17] T. M. Mitchell. *Generalization as search*. Artificial Intelligence, 18(2), 1982.
- [18] C. J. C. H. Watkins and P. Dayan. *Q-learning*. Machine Learning, 8, 1992.
- [19] B. Falkenhainer, K. D. Forbus, and D. Gentner. *The Structure-Mapping Engine: algorithm and examples*. Artificial Intelligence, 41(1), 1989.
- [20] J. P. Marques-Silva and K. A. Sakallah. *GRASP: a search algorithm for propositional satisfiability*. IEEE Transactions on Computers, 48(5), 1999.
- [21] L. De Raedt, A. Kimmig, and H. Toivonen. *ProbLog: a probabilistic Prolog and its application in link discovery*. In: Proceedings of IJCAI, 2007.
- [22] J. E. Laird, A. Newell, and P. S. Rosenbloom. *Soar: an architecture for general intelligence*. Artificial Intelligence, 33(1), 1987.
- [23] R. A. Kowalski. *Predicate logic as programming language*. In: IFIP Congress, 1974.
- [24] M. Minsky. *A framework for representing knowledge*. MIT AI Laboratory Memo 306, 1974.